

Brain Tumor MRI Dataset

Exploratory Data Analysis Report

1. Introduction

It's a full exploratory data analysis (EDA) of a brain tumor MRI dataset that contains four categories of scans: glioma, meningioma, pituitary tumor, and a "no tumor" class. The goal isn't to build a classifier yet — it's to understand what we're working with: how many images per class, what they look like, whether the files are trustworthy, and whether there's anything hiding in the data that could quietly break a model later (duplicate images, corrupt files, or leakage between the training and testing sets).

The analysis is organized as a walkthrough: build an index of every image, check class balance, look at sample images, measure pixel properties, and finally run a set of data-quality checks. Each section below explains what was done, why it matters, and what the results actually showed.

2. Dataset and Environment Setup

The dataset is organized into two folders — Training and Testing — and inside each of those, four subfolders, one per class (glioma, meningioma, no tumor, pituitary). This folder-per-class structure is a very common convention for image datasets because it lets you infer the label directly from the file's location, without needing a separate labels file.

The notebook loads standard Python tools for this kind of work: pandas and numpy for handling data in table form, matplotlib and seaborn for plotting, Pillow (PIL) for opening and inspecting images, and OpenCV (cv2) for faster pixel-level operations like resizing and computing image statistics.

3. Building a File Index

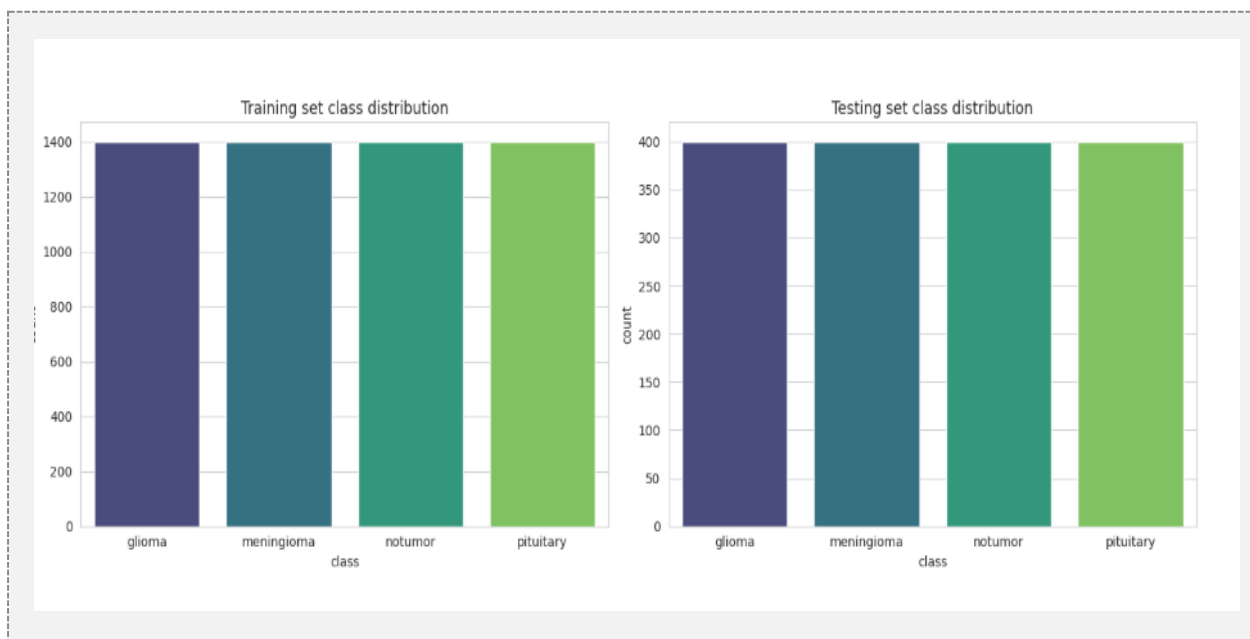
The first real step is to turn a folder structure into something that can actually be analyzed. A small function walks through every class folder in both the Training and Testing directories and records three things for every single image: its file path, its class label, and which split (train or test) it belongs to. All of that gets collected into one big table (a pandas DataFrame) called `df`, plus two smaller tables, `df_train` and `df_test`, for convenience.

Once built, this index confirmed the dataset contains 7,200 images in total across both splits. Having this single table is what makes every later step possible — grouping by class, filtering by

split, computing statistics per category — because everything can now be done with normal table operations instead of digging through folders by hand.

4. Class Distribution (Train vs. Test)

The very first thing worth checking in any classification dataset is whether the classes are balanced. If one class has ten times more images than another, a model can get a deceptively high accuracy just by leaning toward the majority class, without actually learning to tell the classes apart. So this section counts how many images fall into each class, separately for the training set and the testing set, and visualizes it as two side-by-side bar charts.



The exact counts, produced right after the chart, came out as follows:

Split	Glioma	Meningioma	Notumor	Pituitary	Total
Train	1,400	1,400	1,400	1,400	5,600
Test	400	400	400	400	1,600
Total	1,800	1,800	1,800	1,800	7,200

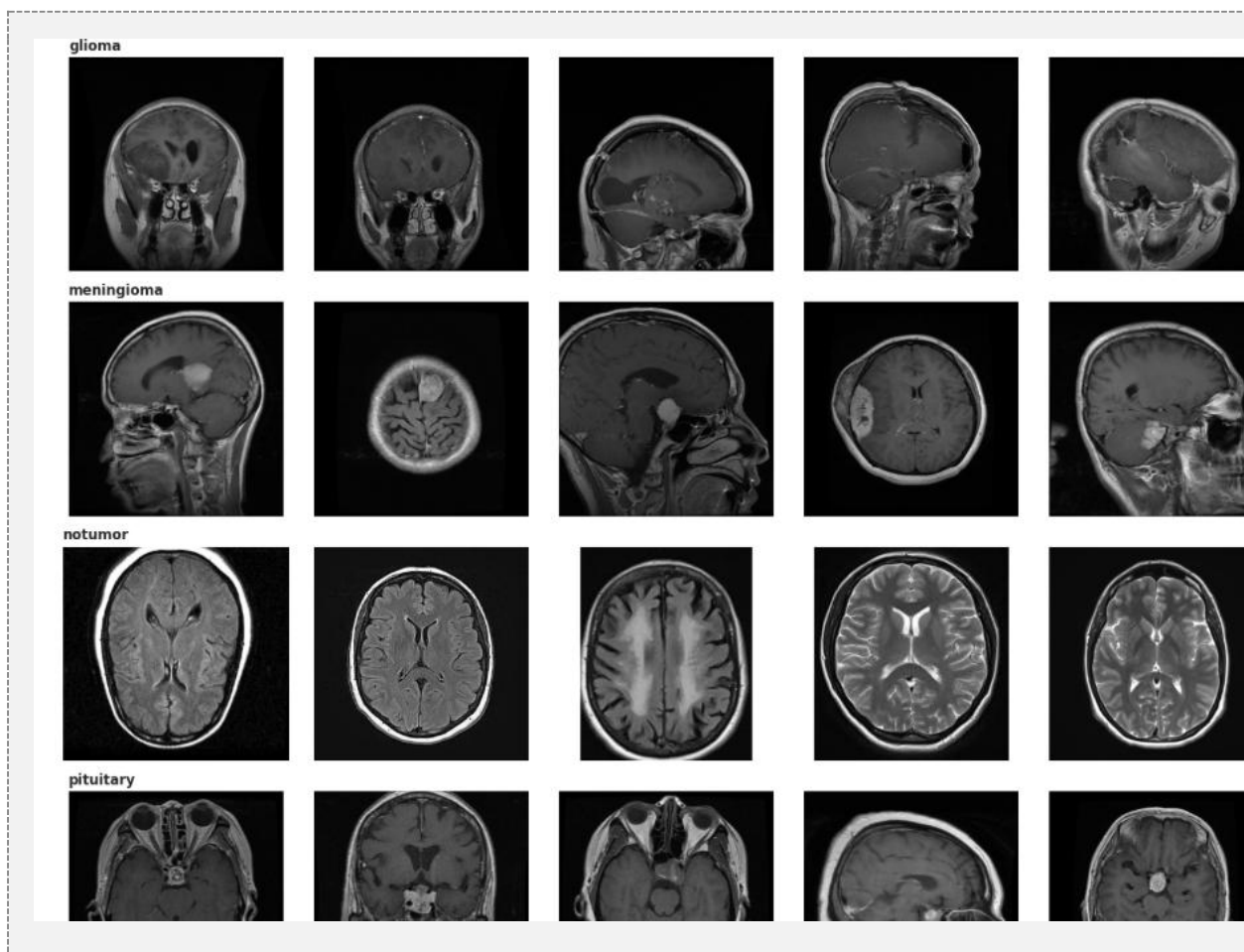
4.1 Quantifying Class Imbalance

Rather than just eyeballing the bar charts, this step puts a number on the balance: for each split, it computes what percentage of the data each class makes up, and the ratio between the largest and smallest class (the “imbalance ratio”). A ratio of 1:1 means perfectly balanced; anything noticeably higher signals a class imbalance problem.

In this dataset, every single class sits at exactly 25% of its split, both in training and testing, giving an imbalance ratio of 1.0 : 1 in both cases. In plain terms — this is about as balanced as a dataset can get. That's a genuinely good sign, because it means no class weighting, oversampling, or synthetic augmentation is needed just to fix imbalance before training a model.

5. Sample Images per Class

Numbers only tell part of the story, so this section pulls five random sample images from each of the four classes (using a fixed random seed so the same samples show up every time the notebook runs) and displays them in a 4-row by 5-column grid, one row per class. This is a simple but important sanity check — it lets a human eye confirm that the images actually look like what their folder claims they are, and gives a first visual impression of how similar or different the tumor types appear on an MRI scan.



6. Image Dimensions and Color Mode

Real-world image datasets are almost never perfectly uniform, so this step opens every one of the 7,200 images and records its width, height, and color mode (for example, RGB for color images or L for grayscale). This matters because most deep learning models expect every input image to be exactly the same size and the same number of color channels — so knowing how much the raw data varies tells us exactly what preprocessing will be required later.

The measurements showed a fair amount of variation:

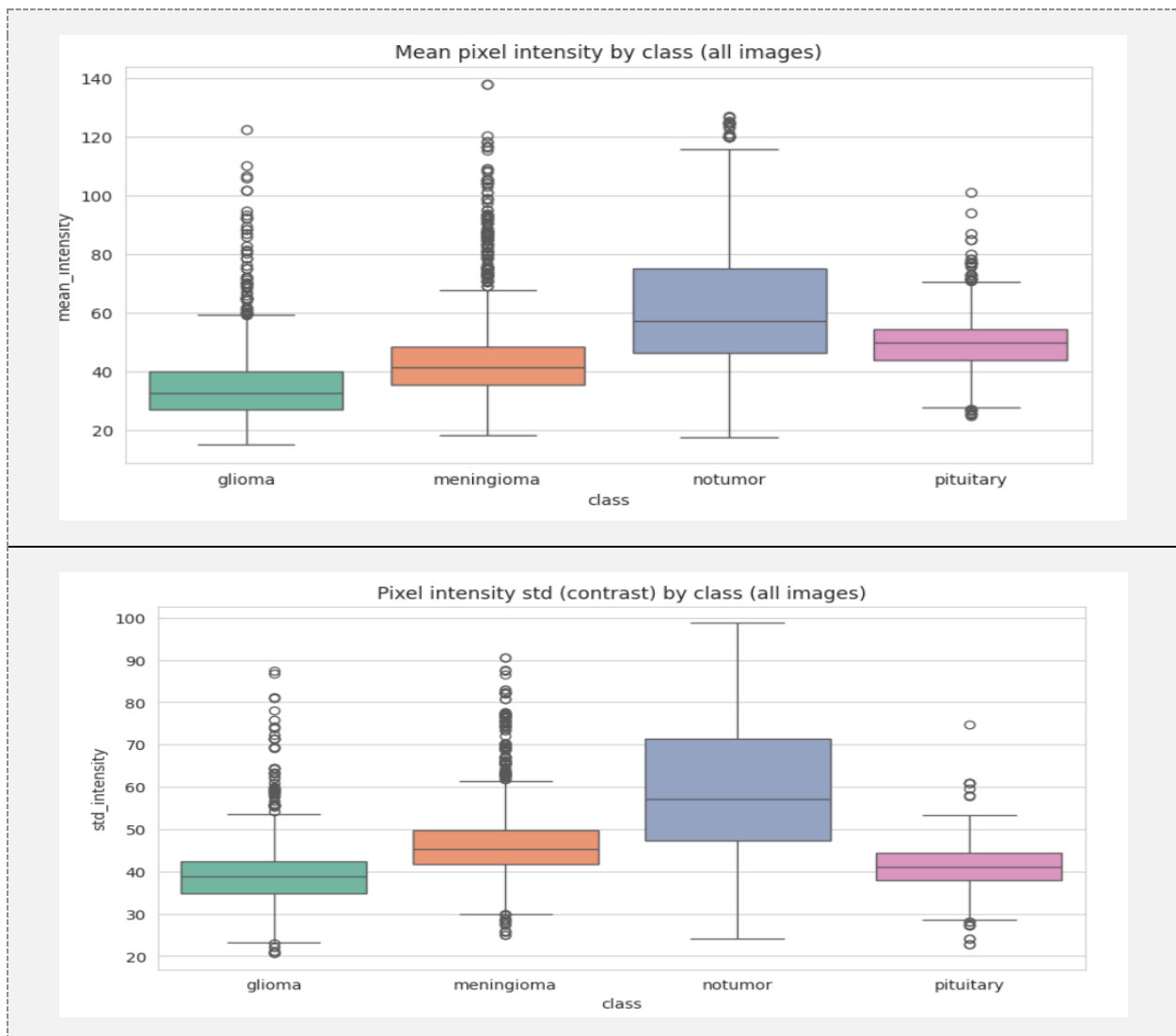
Metric	Width (px)	Height (px)
Mean	453.3	456.9
Std. Dev.	130.0	124.4
Minimum	150	167
Median	512	512
Maximum	1375	1446

So image sizes range from as small as 150×167 pixels up to as large as 1375×1446 pixels, with most images clustering around 512×512. Color mode was mixed too: 4,129 images are stored as RGB (three color channels), 3,067 are grayscale (single channel, mode “L”), and there are a handful of odd ones out — 3 RGBA images and 1 palette-based (“P” mode) image. Since MRI scans don't actually carry color information, the RGB versions are most likely just grayscale scans duplicated across three identical channels — but that assumption needs to be verified rather than taken for granted before collapsing everything to one channel.

7. Pixel Intensity and Contrast by Class

This section digs one level deeper than raw dimensions and looks at what the pixels themselves are actually doing. Every image is resized down to a standard 128×128 grayscale version, and then two numbers are computed: the mean pixel intensity (basically, how bright the image is on average) and the standard deviation of pixel intensity (which acts as a rough proxy for contrast — a high standard deviation means the image has a strong mix of very dark and very bright areas, while a low one means it's fairly flat and uniform).

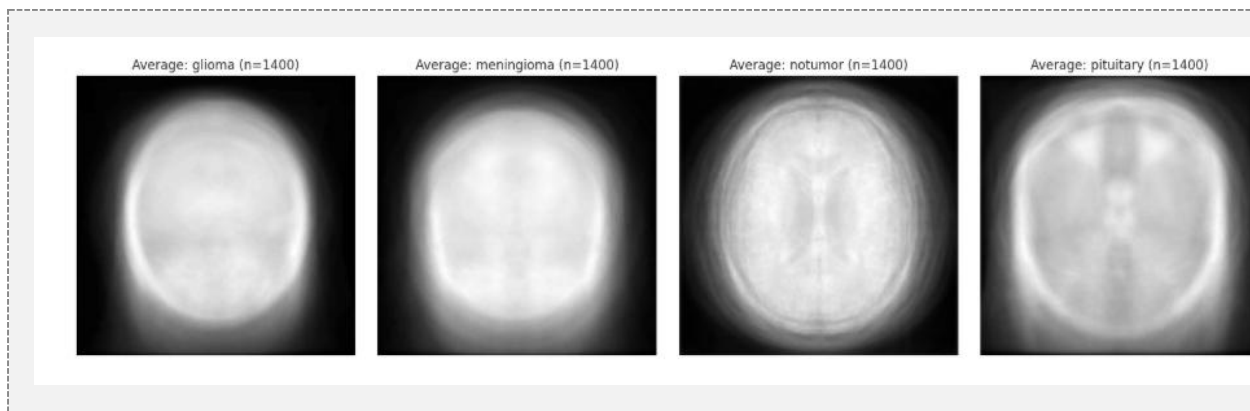
These values are plotted as boxplots, split out by class, so it's easy to see whether different tumor types tend to look brighter, darker, or higher-contrast than each other on average.



8. Average Image per Class

This is one of the more visually interesting checks. For each class, every single training image (not just a sample — all 1,400 of them per class) is resized to 128×128 and stacked together to compute a pixel-by-pixel average, producing one “average brain” image per class. Any feature that shows up consistently in a class — like a bright blob roughly where a certain tumor type tends to appear — will still be visible in the average, while random, non-repeating details from individual scans will blur away.

It's a quick way to see, at a glance, whether there's a class-specific visual signature a model might latch onto, and whether the images across a class are reasonably well-aligned (centered, similarly cropped) or all over the place.



9. Data Quality: Corrupt Files and Duplicates

Before trusting a dataset, it's worth checking for two very practical problems: files that are broken (won't actually open) and files that are exact duplicates of each other. A corrupt file will crash training or silently get skipped, and duplicate images sitting in the training set effectively give that exact picture more “votes” than it should have, which can quietly bias the model toward whatever that duplicated image happens to show.

Every image was checked for corruption using Pillow's built-in verification, and every image was also fingerprinted with an MD5 hash of its raw file bytes so that any two files with an identical hash could be flagged as duplicates.

The results:

- Corrupt files found: 0 — every single image opens cleanly.
- Duplicate images found: 187 — 187 images in the dataset are byte-for-byte identical copies of another image already in the dataset.

Zero corrupt files is a clean bill of health. The 187 duplicates, though, are a real issue worth fixing — leaving exact copies in the training data effectively gives them extra weight during training, and if a train/validation split gets carved out later without accounting for this, an identical image could end up in both the training and validation portions, making validation performance look better than it really is.

10. Train/Test Leakage Check

This check answers a very specific and very important question: does the same exact image ever appear in both the training set and the testing set? If it does, that's called data leakage, and it's a serious problem — a model could effectively “memorize” an image during training and then get

tested on that same image, making its test accuracy look better than its real-world performance would actually be.

Using the same MD5 hashes computed in the previous step, the notebook compares the set of hashes in the training split against the set of hashes in the testing split. The result: 0 images are shared between train and test. This is good news — it means the test set is genuinely unseen data, so any accuracy numbers a future model reports on it can actually be trusted.

11. Missing Values and Label Sanity Check

This is a broader housekeeping pass that checks for the kinds of small mistakes that are easy to overlook but can break a training pipeline. Specifically, it checks: whether any row is missing a filepath, class, or split value; whether any class label exists outside the expected four (glioma, meningioma, notumor, pituitary) — which would catch a misspelled folder name; whether any split label exists outside “train” or “test”; whether every filepath on record actually points to a real file on disk; whether any file is zero bytes (which would open without technically erroring, but contain no actual image data); and whether every class shows up in both splits with the expected number of images.

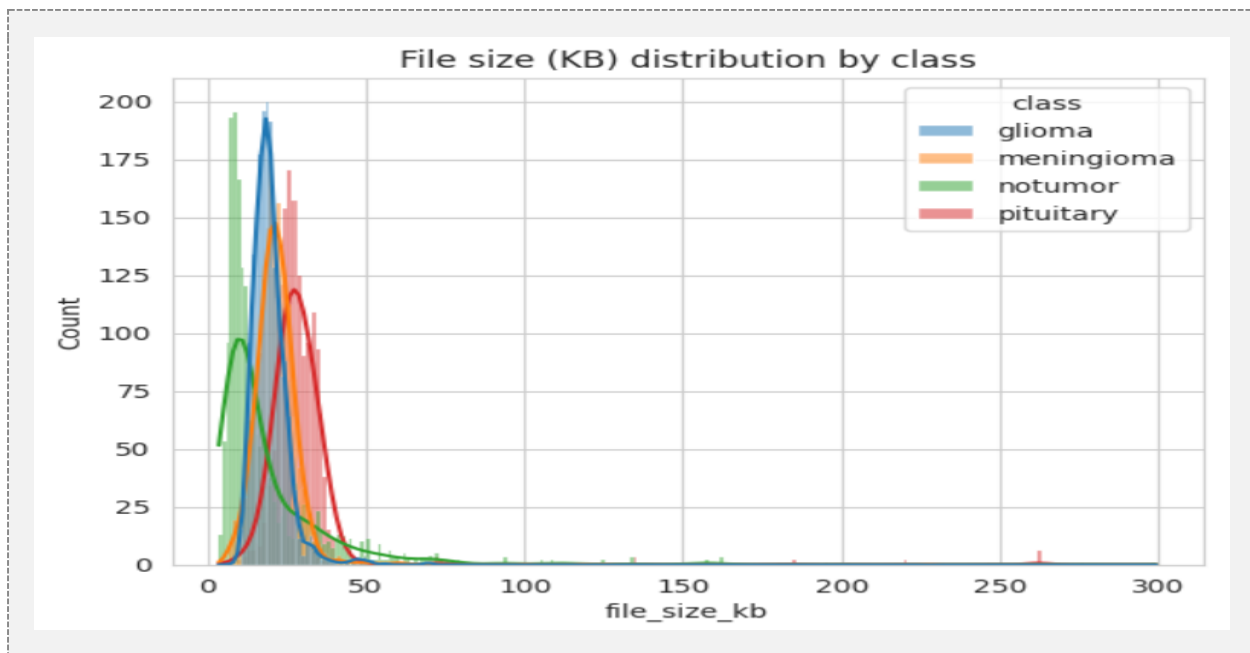
Every single one of these checks came back clean:

- No missing values in filepath, class, or split columns.
- No unexpected class labels.
- No unexpected split labels.
- No records pointing to files that don't exist.
- No zero-byte files.
- Every class present in both splits at the expected counts (1,400 in train, 400 in test).

In short, the labeling and folder structure of this dataset is trustworthy — there's no cleanup needed on that front.

12. File Size Distribution

The last check looks at file size on disk (in kilobytes) for every image, broken down by class. File size is a rough, indirect signal — it's influenced by image resolution, compression, and how much fine detail or noise is in the picture — but wildly different file-size patterns between classes can sometimes hint at systematic differences in how the images were captured or processed.



13. Overall Conclusions and Next Steps

Pulling everything together, here's the state of this dataset:

Class Balance

Perfectly balanced. Every class has exactly 1,400 images in training and 400 in testing (1,800 total per class), for a 1.0 : 1 imbalance ratio in both splits. No class weighting, oversampling, or balance-driven augmentation is required.

Image Consistency

Not consistent — resizing will be required before training. Dimensions range from 150×167 up to 1375×1446 pixels (mean around 453×457, median 512×512), and color mode is mixed (4,129 RGB, 3,067 grayscale, plus a few RGBA/palette outliers). Two fixes are needed, not one: (1) resize every image to a single target resolution — 224×224 is the standard choice if a pretrained CNN backbone will be used, while 512×512 is an option for a from-scratch model; and (2) standardize every image to a single, consistent channel format, most likely grayscale, since MRI scans don't carry real color information. It's worth confirming that the RGB images really are just grayscale duplicated across three channels before collapsing them, rather than assuming it.

Data Quality

Clean on the leakage front, but one issue to fix on duplicates. Corrupt files: 0. Train/test leakage: 0 shared images — test metrics from a future model can be trusted. However, 187 duplicate

images were found (by exact file hash), all within the same split rather than across splits, so these need to be de-duplicated before training. Leaving exact copies in the data gives them disproportionate weight, and if any of them later end up split across a train/validation partition, the model could effectively be validated on an image it already memorized during training.

Label Integrity

Fully confirmed clean. No null filepaths, classes, or splits; no unexpected class or split labels; no missing files; no zero-byte files; and every class is represented in both splits at the expected counts. No cleanup needed here.

Recommended Next Steps

- Remove the 187 duplicate images before any train/validation split is created.
- Resize all images to one consistent target resolution (e.g., 224×224 for a pretrained backbone).
- Standardize all images to a single channel mode (most likely grayscale), after confirming the RGB images are genuinely grayscale content duplicated across channels.
- Once cleaned and standardized, the dataset is otherwise well-balanced and leakage-free, making it a solid, trustworthy foundation for model training.